

TripMateAI - Documentación ampliada de Confluence

1. TripMateAI - Visión general del proyecto

Resumen de la sección Visión de producto, funcionalidades principales, roles y arquitectura global del sistema.

Tareas relacionadas TRIP-7, TRIP-8, TRIP-9, TRIP-11, TRIP-14, TRIP-17, TRIP-26, TRIP-75, TRIP-76, TRIP-77

Enlaces de Confluence Progresión del proyecto

1. Propósito

TripMateAI es una plataforma integral de planificación de viajes impulsada por Inteligencia Artificial.

Solucionamos el problema de la fragmentación en el turismo (tener que visitar múltiples webs), permitiendo a los usuarios gestionar todo (vuelos, hoteles, itinerarios) en un solo lugar.

Nuestra propuesta de valor única es un Chatbot con IA capaz de generar itinerarios completos ajustados a un presupuesto específico (ej: "Prepárame un viaje a París por 1000€") y a las preferencias personales del usuario.

2. Funcionalidades Clave y Estado

Aquí mostramos el progreso actual de las funcionalidades principales vinculado a nuestro backlog en Jira:

- **Planificador "Todo en Uno":** Búsqueda centralizada de vuelos, hoteles y actividades sin salir de la aplicación. (Historias relacionadas: TRIP-8 y TRIP-17)
- **Agente de Viajes con IA:** Chatbot inteligente para la creación automática de itinerarios y gestión de presupuestos. (Historias relacionadas: TRIP-9 y TRIP-14)
- **Experiencia de Usuario Personalizada:** Interfaz intuitiva y perfiles personalizados con gestión de medios. (Historias relacionadas: TRIP-26 y TRIP-11)

3. Arquitectura de Alto Nivel

El sistema utiliza una arquitectura híbrida optimizada para rendimiento y seguridad:

- **Frontend (Angular):** Gestiona la autenticación directamente con Firebase para rapidez.

- **Backend (Node.js):** Centraliza la lógica de negocio compleja, las conexiones con la IA y protege las llamadas a APIs externas.
- **BaaS (Firebase):** Ecosistema para Auth, Base de Datos (Firestore) y Almacenamiento de archivos (Storage).
- **Inteligencia Artificial (Chatbot):** Módulo encargado de procesar los prompts de viaje y generar respuestas estructuradas en JSON.

4. Arquitectura Global del Sistema

- **Guía de Desarrollo:** Guía paso a paso para nuevos desarrolladores (instalación, variables de entorno y despliegue).
- **App Móvil (Android):** Versión nativa desarrollada en Kotlin y Jetpack Compose, conectada a la misma API que la web.
- **Progresión del proyecto:** Resumen de la evolución del proyecto con actividades.

2. Guía de desarrollo

Resumen de la sección Requisitos previos, comandos de instalación y variables de entorno necesarias para ejecutar el proyecto.

Tareas relacionadas TRIP-7, TRIP-21

1. Requisitos Previos

- **Node.js:** (Versión recomendada, v20.15.0 o superior).
- **Angular CLI:** Instalado globalmente (**`npm install -g @angular/cli`**).
- **Bruno:** Cliente para probar las APIs (Descargar Bruno).
- **Android Studio:** Ver historia de configuración: TRIP-21.

2. Instalación y Comandos

Clona el repositorio y ejecuta los siguientes comandos en tu terminal:

- **`npm install`**
- **`npm install -g @angular/cli`**
- **`npm run dev / npm start`**
- **`npm run build`**
- **`npm test`**

3. Variables de Entorno (.env)

Crea un archivo `.env` en la raíz del servidor. **IMPORTANTE:** Estas claves son secretas, no las compartas en el repositorio.

Variable	Descripción
Configuración de la App	
PORT	Puerto donde corre el servidor (ej: 3000)
NODE_ENV	Entorno de ejecución (development o production)
JWT_SECRET	Clave secreta para firmar tokens
BCRYPT_SALT_ROUNDS	Nivel de complejidad para encriptar contraseñas

Base de Datos

DATABASE_URL URL de conexión a la base de datos (si aplica)

Firebase Admin SDK Credenciales para que Node.js se conecte a Firebase

FIREBASE_PROJECT_ID ID del proyecto en la consola de Firebase

FIREBASE_CLIENT_EMAIL Email de la cuenta de servicio

FIREBASE_PRIVATE_KEY (Sensible) Llave privada del Admin SDK

FIREBASE_PRIVATE_KEY_ID ID de la llave privada

Configuración Cliente Firebase Configuración pública para el Frontend

FIREBASE_API_KEY API Key pública

FIREBASE_AUTH_DOMAIN Dominio de autenticación

FIREBASE_APP_ID ID de la Aplicación

3. Arquitectura Backend y API

Resumen de la sección Stack backend, modelo de datos Firestore y endpoints REST documentados para usuarios, perfiles, viajes, actividades, media y autenticación.

Tareas relacionadas TRIP-7, TRIP-17, TRIP-25, TRIP-26, TRIP-54

1. Pila Tecnológica (Stack)

- Servidor: Node.js
- Base de Datos: Google Firestore (NoSQL)
- Autenticación: Firebase Auth (Email/Contraseña y Google)
- Pruebas de API: Bruno/Swagger

2. Modelo de Datos (Firestore)

Estructura de las colecciones principales en la base de datos:

- **users:** Información básica de la cuenta.
- **profiles:** Datos extendidos del perfil (preferencias, avatar). TRIP-26
- **trips:** Datos generales de un viaje (destino, presupuesto, fechas). TRIP-17
- **activities:** Detalles de las actividades dentro de un itinerario. TRIP-54
- **media:** Imágenes a subir de usuarios. TRIP-26

3. Endpoints de la API (Documentación en Bruno/Swagger)

Las peticiones HTTP se gestionan y prueban a través de Bruno y Swagger. La estructura sigue estas rutas:

Users (Usuarios):

- GET /api/users/ - Listado de usuarios.
- GET /api/users/{id} - Usuario específico por ID.
- DELETE /api/users/{id} - Eliminar usuario específico por ID.
- GET /api/profiles - Obtener perfiles de usuarios.
- GET /api/profiles/{id} - Perfil específico por ID.
- DELETE /api/profiles/{id} - Eliminar perfil específico por ID.

Trips (Viajes):

- POST /api/trips - Crear un nuevo viaje.
- GET /api/trips - Listar mis viajes guardados.
- GET /api/trips/{id} - Viaje específico por ID.
- PUT /api/trips/{id} - Editar viaje por ID.
- DELETE /api/trips/{id} - Eliminar viaje específico por ID.

Activities (Actividades):

- GET /api/activities - Buscar actividades disponibles.

- POST /api/activities - Crear actividades.
- PUT /api/activities/{id} - Editar actividad por ID.
- DELETE /api/activities/{id} - Eliminar actividad específica por ID.

Media (Imágenes):

- POST /api/media/upload - Enviar imágenes.
- PUT /api/media/upload/{id} - Editar imagen.

Auth (Autenticación):

- POST /api/auth/register - Registrar un usuario nuevo.
- PUT /api/auth/login - Inicio de sesión de un usuario ya registrado.

4. Criterio de seguridad

La documentación de Confluence plantea que el backend centraliza la lógica sensible y evita que el frontend acceda directamente a credenciales privadas o servicios externos críticos.

4. Arquitectura Frontend (Angular)

Resumen de la sección Tecnologías frontend, organización del código y comunicación entre Angular, Firebase y Node.js.

Tareas relacionadas TRIP-7, TRIP-12, TRIP-17, TRIP-20, TRIP-26, TRIP-54

1. Tecnologías Principales

Estas son las herramientas que usamos para construir la parte visual de TripMateAI:

- **Angular (v19):** El motor principal. Usamos la versión más moderna para que la web sea rápida.
- **Tailwind CSS:** Para diseñar los estilos. Nos permite maquetar botones, colores y formas rápidamente sin escribir archivos CSS gigantes.
- **Angular Signals:** Es nuestro sistema de "reacción". Si cambia un dato (ej: el precio de un viaje), Signals se encarga de actualizar solo ese número en la pantalla instantáneamente.

2. Organización del Código (src/app)

Para no perdernos, organizamos los archivos según su función en la aplicación:

core/ Aquí guardamos lo que hace que la app funcione por detrás. No tiene "vistas", solo lógica.

- **Servicios:** Son los encargados de hablar con el Servidor.
- **Guardias (Guards):** Son los encargados de hacer que si no estás logueado, no te dejan entrar a /dashboard.

pages/ Aquí están las vistas completas que visita el usuario. Cada carpeta es una pantalla diferente vinculada a nuestras tareas de Jira:

- **dashboard/:** La portada pública. TRIP-12
- **auth/:** Pantallas de Login y Registro. TRIP-7
- **trips/:** Donde el usuario maneja sus viajes. TRIP-17
- **profile/:** Donde el usuario edita sus datos y sube su foto. TRIP-26
- **activities/:** La pantalla del chat con la IA. TRIP-54

shared/ Aquí guardamos los componentes que reutilizamos en muchas pantallas para no repetir código.

- **Sidebar:** La barra de navegación lateral. TRIP-20

3. ¿Cómo se comunican las partes?

Nuestro Frontend (Angular) intenta ser lo más sencillo posible y delega el trabajo pesado al Servidor.

A. Autenticación (Login)

- Directa: Angular habla directamente con Google/Firebase para iniciar sesión.
- ¿Por qué? Es más rápido y seguro que manejar contraseñas nosotros mismos.

B. Datos y Viajes

- A través de Node: Angular nunca toca la base de datos directamente.
- Angular le pide los viajes al Servidor Node (GET /api/trips), el servidor verifica quién eres, busca los datos y te los devuelve.

C. Imágenes (Fotos de Perfil/Viaje)

- A través de Node (Proxy): El usuario selecciona la foto en Angular. Angular envía el archivo físico al Servidor Node (mediante una petición POST o PUT). El Servidor Node recibe el archivo, lo sube a la nube (Firebase Storage) y guarda la referencia.
- Ventaja: Angular no necesita saber claves de almacenamiento, solo "envía el archivo" y se olvida.

5. Infraestructura Firebase

Resumen de la sección Uso de Firebase Auth, Firestore y Storage dentro de una arquitectura híbrida con Node.js como middleware seguro.

Tareas relacionadas TRIP-7, TRIP-17, TRIP-23, TRIP-25, TRIP-26

1. Visión General

En TripMateAI, utilizamos Firebase como nuestro proveedor de Backend (BaaS) para gestionar la identidad, la persistencia de datos y el almacenamiento de archivos.

Nuestra arquitectura es Híbrida:

- **Angular (Cliente):** Habla directamente con Firebase solo para Autenticación.
- **Node.js (Servidor):** Actúa como middleware seguro. Utiliza el Admin SDK para leer/escribir en Firestore, permitiéndonos ejecutar lógica de negocio y validaciones antes de tocar la base de datos.

2. Autenticación (Firebase Auth)

Gestionamos la identidad de los usuarios delegando la seguridad en Google.

- Métodos Habilitados: Email / Contraseña (Registro estándar) y Google Sign-In.
- Estado de Implementación: TRIP-25 (Configuración de consola). TRIP-7 (Integración en Angular).

Nota de Seguridad: Aunque el login se hace en el frontend, cada petición al servidor Node.js debe llevar el ID Token en la cabecera Authorization. El servidor verifica este token con Firebase antes de responder.

3. Base de Datos (Firestore)

Utilizamos una base de datos NoSQL.

- Modo de Conexión: Lectura/Escritura (Backend) vía firebase-admin (Privilegios de superusuario). Lectura (Frontend) limitada.
- La mayoría de datos se sirven a través de nuestra API REST en Node.js, no por conexión directa.

4. Almacenamiento (Storage)

Repositorio de objetos para contenido multimedia (imágenes de perfil, portadas de viajes).

- Estructura de Buckets: /profile/{uid}/: Archivos privados del usuario. /trips/{tripId}/: Archivos compartidos del viaje.
- Implementación: TRIP-26 (Subida de avatar).

6. Inteligencia Artificial (chatbot)

Resumen de la sección Flujo seguro de IA, ingeniería de prompts y reglas para generar itinerarios ajustados al presupuesto.

Tareas relacionadas TRIP-9, TRIP-14

1. Estado de la Integración

Desarrollo activo de las capacidades cognitivas de la aplicación.

- Infraestructura: Configuración del cliente y gestión de claves API. TRIP-14
- Lógica de Negocio: Generación de itinerarios basados en inputs del usuario. TRIP-9

2. Flujo de Interacción Seguro

Por razones de seguridad, el Frontend nunca comunica directamente con la IA.

- Angular: Recoge los inputs (Destino: "Paris", Presupuesto: "1000€").
- Node.js: Recibe los datos. Construye el Prompt dinámico. Inyecta la API KEY (variable de entorno). Envía la petición al proveedor (OpenAI/Gemini).
- Validación: Node.js recibe la respuesta, verifica que sea un JSON válido y lo envía al Frontend.

3. Ingeniería de Prompts (Prompt Engineering)

El prompt se construye en tiempo de ejecución concatenando las instrucciones del sistema con los datos del usuario.

A. System Prompt (Instrucción Base) Esta instrucción es invisible para el usuario, pero define la personalidad y reglas del bot.

"Eres un experto agente de viajes de TripMateAI. Tu objetivo es crear itinerarios detallados y realistas. Ajustate estrictamente al presupuesto indicado. Tu respuesta debe ser exclusivamente un objeto JSON válido, sin texto introductorio. Si el presupuesto es irreal (ej: 10€ para viajar a Japón), devuelve un JSON con un campo de error."

Reglas del prompt base

- Eres un experto agente de viajes de TripMateAI.
- Ajustate estrictamente al presupuesto indicado.
- Tu respuesta debe ser exclusivamente JSON valido.
- Si el presupuesto es irreal, devuelve un JSON con un campo de error.

7. App móvil (Android)

Resumen de la sección Stack móvil con Kotlin, Jetpack Compose, Retrofit, Coil y arquitectura MVVM.

Tareas relacionadas TRIP-75, TRIP-76, TRIP-77

1. Stack Tecnológico

- Lenguaje: Kotlin
- UI: Jetpack Compose (Material Design 3)
- Networking: Retrofit
- Imágenes: Coil (Carga asíncrona desde URLs de Firebase)

2. Arquitectura (MVVM)

- Data Layer: Repositorios que hablan con la API de Node.
- UI Layer: ViewModels y Composables.

8. Reserva de vuelos

Resumen de la sección Flujo de reserva de vuelos desde búsqueda y selección de asiento hasta datos del pasajero y pago.

Tareas relacionadas TRIP-63, TRIP-64, TRIP-65, TRIP-66, TRIP-67

1. Contenido

Implementación de un flujo completo de reserva de vuelos, que permite al usuario buscar, seleccionar, personalizar y pagar un itinerario aéreo dentro de la plataforma TripMateAI.

2. Flujo de Usuario

El proceso sigue un modelo lineal de "Embudo de Conversión" de 4 pasos.

3. Requerimientos Funcionales por Pantalla

A. Buscador de Vuelos

- Inputs: Origen (Aeropuerto), Destino (Aeropuerto), Fecha Salida, Pasajeros (1-9).
- Selector de Clase: 3 opciones: Económica, Trabajo, Primera Clase.

B. Selección de Asientos

- Visualización gráfica del avión.
- Diferenciación visual entre asientos: Libre, Ocupado, Seleccionado.
- Lógica: El precio varía según la clase seleccionada en el paso anterior.

C. Información del Pasajero

- Formulario reactivo.
- Campos: Nombre, Apellido, DNI/Pasaporte, Email.
- Validación estricta (no permitir avanzar si faltan datos).

D. Pasarela de Pago

- Resumen del pedido (Vuelo + Asiento + Impuestos).
- Integración simulada de pago (Tarjeta Crédito).

9. Traducción

Resumen de la sección Sistema de traducción dinámica con español, inglés e italiano, persistiendo la preferencia del usuario.

Tareas relacionadas TRIP-72, TRIP-73, TRIP-74

1. Contenido

Implementación de un sistema de traducción dinámica que permite cambiar el idioma de la interfaz en tiempo real sin recargar la página.

2. Idiomas Soportados

- Español (Default): es.json
- Inglés: en.json
- Italiano: it.json

3. Arquitectura Técnica

- Almacenamiento: Persistencia de la preferencia del usuario en localStorage (Si el usuario elige Italiano, la próxima vez que entre, se mantendrá en Italiano).

10. Progresión del proyecto

Resumen de la sección Página de seguimiento que enlaza con el resumen del proyecto en Jira.

Enlaces de Confluence

[Progresión del proyecto](#)

11. Documentación del proyecto

Resumen de la sección [Referencia a la documentación publicada del proyecto.](#)